# JAX for numpy users

Colm Talbot (colm.talbot@princeton.edu)

Princeton University Research Computing

March 6, 2026

PRINCETON UNIVERSITY

# What is JAX?

## High performance array computing

JAX is a Python library for accelerator-oriented array computation and program transformation, designed for high-performance numerical computing and large-scale machine learning.

https://docs.jax.dev/en/latest/

# JAX fundamentals

## Familiar API

JAX provides a familiar NumPy-style API for ease of adoption by researchers and engineers.

## Transformations

JAX includes composable function transformations for compilation, batching, automatic differentiation, and parallelization.

## Run anywhere

The same code executes on multiple backends, including CPU, GPU, & TPU

https://docs.jax.dev/en/latest/

# Why should I use JAX?

**“I want my numpy code to run on a GPU”**

**“I want to calculate derivatives without doing algebra”**

**“I want my numpy code to run in parallel”**

**“I want my numpy code to run faster”**

# Am I already using JAX?

**LLMs: Gemini/Gemma (Google), Claude (Anthropic), Grok (xAI), Apple, ...**

**Deep learning: equinox, flax, ...**

**Bayesian methods: numpyro, blackjax, ...**

**...**

# How does JAX work?

**JAX operations are:**

- **compiled on demand for whatever hardware you are using with OpenXLA**
- **automatically vectorizable to enable batched computation**
- **differentiable using automatic differentiation**

**Most widely used functionality is exposed via a numpy-like API**

# First interactive break

## JAX, numpy, and scipy

# Digression: the Python array API standard – as easy as xp

**Problem: JAX is not the only package to implement a numpy-like API (cupy, torch, dask, …) and each has their own quirks**

**Solution: Make everyone agree on a common set of functionality a Python array should be able to do, a.k.a., a common Application Programming Interface (API)**

```
import numpy as xp
import jax.numpy as xp
```

https://data-apis.org/array-api/latest/index.html

# Digression: the Python array API standard – as easy as xp

**Downstream libraries are adopting compatibility (e.g., scipy)**

**A supporting ecosystem is developing to enable frictionless use in Python projects:**

- **What array backend am I using (array-api-compat)?**
- **What if I need functionality beyond the standard (array-api-extra)?**
- **How do I maintain support for type checking (array-api-typing)?**

https://data-apis.org/array-api/latest/index.html

# Second interactive break
# when is jax.numpy not numpy?

# Beyond jax.numpy – JIT compilation

JAX uses the accelerated linear algebra (XLA) compiler to optimize the evaluation of functions.

The compilation is performed the first time a function is evaluated and is specific to the shapes and types of the function arguments.

XLA "compiles away" some intermediate arrays and operations that don't impact the result, e.g., print statements.

There are important limitations on what operations are permitted in JIT-compiled functions and what the arguments can be.

# Third interactive break

JIT

# JIT compilation – summary

JIT compilation is specific to the shape of the input arrays

Control flow (e.g., if statements) can only depend on things that are known at compile time, e.g., static arguments and array shapes

Code that doesn't impact the result values will be removed at the compilation stage, including print statements, use jax.debug.print if you need to inspect variables within compiled functions.

# Beyond jax.numpy – autodiff

**Automatic differentiation (autodiff) produces exact derivatives of arbitrary functions as long as they are composed of operations with well defined derivatives by repeated application of the chain rule and is fundamental to deep learning.**

**Within JAX forward and reverse mode autodiff are implemented. The distinction is beyond today's scope, but there is plenty of information in the JAX documentation.**

# Fourth interactive break

## autodiff

# Beyond jax.numpy – vectorization

**In numpy, vectorization is achieved using serial loops written in C for performance and compiled in advance.**

**In JAX, JIT compilation means that vectorized functions evaluate in parallel to whatever degree is possible on the provided hardware.**

```
import jax
jax.vmap(function)
```

# Fifth interactive break

# vectorization

# What did I miss?

Sharding, i.e., multi-device computation

JAX-native control flow, e.g., jax.lax.scan, jax.lax.cond

Benchmarking and profiling

Ahead of time compilation

The wider JAX ecosystem

# Summary

**JAX is the foundation of many machine learning frameworks and an increasingly important part of the scientific software stack**

**jax.numpy can be used as a near drop-in replacement for numpy yielding instant parallelisation**

**The JAX autodifferention functionality is extremely flexible and composable with other JAX transformations**

**There is a flourishing ecosystem of packages using JAX!**